

Welcome to Week10

Module 10 tutorial preparation

In this tutorial, we will learn about SQL and databases

Step 1: Install dbBrowser:

- [Downloads](#)  (dbBrowser, 2023)

Step 2: Download the practice database:

- [DreamHome.db](#)  (UC, 2023)

Challenge 1: Retrieve all columns, all rows

- List full details of all staff. Because there are no restrictions specified in this query, the WHERE clause is unnecessary, and all columns are required. We write this query as:

```
SELECT staffNo, fName, lName, oPosition, sex, DOB, salary, branchNo  
FROM Staff;
```

```
SELECT *  
FROM Staff;
```

Challenge 2: Retrieve specific columns, all rows

- Produce a list of salaries for all staff, showing only the staff number, the first and last names, and the salary details.

```
SELECT staffNo, fName, lName, salary  
FROM Staff;
```

Challenge 3 Use of DISTINCT

- List the property numbers of all properties that have been viewed.

```
SELECT propertyNo  
FROM Viewing;
```

- The result table is shown in Table 3 above. Notice that there are several duplicates, because unlike the relational algebra Projection, SELECT does not eliminate duplicates when it projects over one or more columns. To eliminate the duplicates, we use the DISTINCT keyword. Rewriting the query as:

```
SELECT DISTINCT propertyNo  
FROM Viewing;
```

Challenge 4: Calculated fields

- Produce a list of monthly salaries for all staff, showing the staff number, the first and last names, and the salary details

```
SELECT staffNo, fName, lName, salary/12  
FROM Staff;
```

```
SELECT staffNo, fName, lName, salary/12 AS monthlySalary  
FROM Staff;
```

Part 2: Row selection (WHERE clause)

- The five basic search conditions (or predicates, using the ISO terminology) are as follows:
- Comparison: Compare the value of one expression to the value of another expression.
- Range: Test whether the value of an expression falls within a specified range of values.
- Set membership: Test whether the value of an expression equals one of a set of values.
- Pattern match: Test whether a string matches a specified pattern.
- Null: Test whether a column has a null (unknown) value.

Challenge 5: Comparison search condition

- List all staff with a salary greater than £10,000.

```
SELECT staffNo, fName, lName, salary  
FROM Staff WHERE salary > 10000;
```

- In SQL, the following simple comparison operators are available:
- = equals
- <> is not equal to (ISO standard)
- != is not equal to (allowed in some dialects)
- < is less than
- <= is less than or equal to
- > is greater than
- >= is greater than or equal to

More complex predicates can be generated using the logical operators AND, OR, and NOT, with parentheses (if needed or desired) to show the order of evaluation.

The rules for evaluating a conditional expression are:

- an expression is evaluated left to right;
- subexpressions in brackets are evaluated first;
- NOTs are evaluated before ANDs and ORs;
- ANDs are evaluated before ORs.
- The use of parentheses is always recommended, in order to remove any possible ambiguities.

Challenge 6: Compound comparison search condition

- List the addresses of all branch offices in London or Glasgow.

```
SELECT *  
FROM Branch  
WHERE city = "London" OR  
city = "Glasgow";
```

Challenge 7 Range search condition (BETWEEN/NOT BETWEEN)

- List all staff with a salary between £20,000 and £30,000.

```
SELECT staffNo, fName, Lname, position, salary  
FROM Staff  
WHERE salary BETWEEN 20000 AND 30000;
```

Part 3: Sorting Results (ORDER BY Clause)

SQL ORDER BY Clause

- Controls the sorting order of query results
- Query results have no specific order by default
- Must be the last clause in a SELECT statement

Sorting Methods

- Can sort by one or multiple columns (comma-separated)
- Supports ascending (ASC) and descending (DESC) order
- Can use column names or position numbers (1,2,3...)

Important Notes

- Can sort by columns not displayed in results
- Some SQL dialects require ORDER BY elements to appear in the SELECT list

Challenge 8 Single-column ordering)

Produce a list of salaries for all staff, arranged in descending order of salary.

```
SELECT staffNo, fName, Lname, salary  
FROM Staff  
ORDER BY salary DESC
```

Challenge 9 Multiple column ordering

Produce an abbreviated list of properties arranged in order of property type.

```
SELECT propertyNo, propertyType, rooms, rent  
FROM PropertyForRent  
ORDER BY propertyType;
```

To arrange the properties in order of rent, we specify a minor order:

Challenge 9 Multiple column ordering

Produce an abbreviated list of properties arranged in order of property type.

```
SELECT propertyNo, propertyType, rooms, rent  
FROM PropertyForRent  
ORDER BY propertyType;
```

To arrange the properties in order of rent, we specify a minor order:

```
SELECT propertyNo, propertyType, rooms, rent  
FROM PropertyForRent  
ORDER BY propertyType, rent DESC;
```

Major


Minor


Part 4: Using the SQL Aggregate Functions

Standard Aggregate Functions

- COUNT - Number of values in a column
- SUM - Total of values in a column (numeric only)
- AVG - Average of values in a column (numeric only)
- MIN - Smallest value in a column
- MAX - Largest value in a column

Part 4: Using the SQL Aggregate Functions

Syntax:

1. COUNT(): Counts the number of rows

`COUNT(*)` -- counts all rows

`COUNT(column_name)` -- counts non-NULL values in a column

`COUNT(DISTINCT column_name)` -- counts unique non-NULL values

2. SUM()/AVG()/MIN()/MAX()

`SUM/MIN/AVG/MAX(column_name)`

Part 4: Using the SQL Aggregate Functions

Syntax:

```
SELECT COUNT(column2)  
FROM table_name
```

Part 4: Using the SQL Aggregate Functions

Key Behaviors

- Functions operate on a single column and return a single value
- All functions except COUNT(*) ignore NULL values
- COUNT(*) counts all rows regardless of NULL or duplicate values

Part 4: Using the SQL Aggregate Functions

DISTINCT Keyword

- Eliminates duplicates before applying the function
- Can affect SUM and AVG results
- Has no effect on MIN and MAX
- Can be specified only once per query

Part 4: Using the SQL Aggregate Functions

Usage Rules

- Aggregate functions can only appear in SELECT list and HAVING clause
- Without GROUP BY, columns in SELECT must be within aggregate functions
- Example of incorrect query:

```
SELECT staffNo,COUNT(salary)  
FROM Staff
```

Challenge 10: Use of COUNT(*)

How many properties cost more than £350 per month to rent?

```
SELECT COUNT(*) AS myCount  
FROM PropertyForRent  
WHERE rent > 350;
```


Challenge 11 Use of COUNT and SUM

Find the total number of Managers and the sum of their salaries.

Challenge 11 Use of COUNT and SUM

Find the total number of Managers and the sum of their salaries.

```
SELECT COUNT (staffNo) AS myCount, SUM (salary) AS mySum  
FROM Staff  
WHERE oPosition = "Manager";
```

Challenge 12 Use of MIN, MAX, AVG

Find the minimum, maximum, and average staff salary.

```
SELECT MIN(salary) AS myMin, MAX (salary) AS myMax, AVG (salary) AS myAvg  
FROM Staff;
```

Part 5: Grouping Results (GROUP BY Clause)

Grouping Query Basics

- GROUP BY clause generates data subtotals
- Grouped queries organize data and produce one summary row per group
- Columns in GROUP BY are called grouping columns

SELECT and GROUP BY Integration Rules

- Each SELECT item must be single-valued per group
- SELECT clause can only contain:
 - Column names
 - Aggregate functions
 - Constants
 - Expressions combining these elements

Part 5: Grouping Results (GROUP BY Clause)

Important Restrictions

- All column names in SELECT must appear in GROUP BY (unless used only in aggregate functions)
- GROUP BY clause may contain columns not listed in SELECT

WHERE and GROUP BY Order

- WHERE conditions are applied first
- Groups are then formed from remaining rows that satisfy the search condition

Challenge 13: Use of GROUP BY

Find the number of staff working in each branch and the sum of their salaries.

```
SELECT branchNo, COUNT (staffNo) AS myCount, SUM (salary) AS  
mySum  
FROM Staff  
GROUP BY branchNo  
ORDER BY branchNo;
```

Part 6: Restricting groupings (HAVING clause)

Purpose and Function

- HAVING clause restricts which groups appear in final results
- Works specifically with GROUP BY clause

HAVING vs WHERE Comparison

- WHERE filters individual rows before grouping
- HAVING filters groups after they are formed

Part 6: Restricting groupings (HAVING clause)

ISO Standard Requirements

- Column names in HAVING must either:
- Appear in the GROUP BY list, or
- Be contained within an aggregate function

Practical Considerations

- HAVING conditions typically include at least one aggregate function
- Without aggregate functions, conditions could be moved to WHERE
- Aggregate functions cannot be used in WHERE clause
- Any query using HAVING can be rewritten without it

Challenge 14: Use of HAVING

For each branch office with more than one member of staff, find the number of staff working in each branch and the sum of their salaries.

```
SELECT branchNo, COUNT (staffNo) AS myCount, SUM (salary)  
AS mySum  
FROM Staff  
GROUP BY branchNo  
HAVING COUNT (staffNo) > 1  
ORDER BY branchNo;
```

Part 7: Multi-table Queries

The queries we've considered so far have a major constraint - all columns in the result table must come **from a single table**. This is often insufficient for answering common user queries.

Purpose of **joins**: **Join** operations allow you to combine information from multiple tables into one result table by forming pairs of related rows.

How joins work: **Joins combine row pairs where matching columns in both tables have the same value.**

Part 7: Multi-table Queries

Basic syntax: To perform a join, include multiple table names in the FROM clause separated by commas, typically using a WHERE clause to specify the join columns.

For example:

SELECT columns

FROM table1, table2

WHERE table1.column = table2.column;

Part 7: Multi-table Queries

Table Aliases

You can use aliases for tables in the FROM clause. The alias is separated from the table name with a space. Aliases can qualify column names to avoid ambiguity and provide shorthand notation. If an alias is provided, it can be used anywhere instead of the table name.

For example:

SELECT columns

FROM table1 t1, table2 t2

WHERE t1.column = t2.column;

Challenge 15: Simple join

List the names of all clients who have viewed a property, along with any comments supplied.

```
SELECT c.clientNo, fName, Lname, propertyNo, comments  
FROM Client c, Viewing v  
WHERE c.clientNo = v.clientNo;
```

Primary Key Vs. Foreign Key

Primary Key

A primary key is a field or combination of fields in a table that **uniquely identifies each record** in that table. It cannot contain duplicate values or null values.

In this example:

- `clientNo` in the Client table is the primary key
- Each client has a unique client number; no two clients can share the same `clientNo`
- Using the primary key, we can locate precisely one specific client record

Primary Key

Characteristics of primary keys:

- 1.Uniqueness: **No two records in the table** can have the same primary key value
- 2.Non-null: Primary keys **cannot be NULL**
- 3.Minimality: **Only necessary columns** are included in the primary key

Foreign Key

A foreign key is a field or combination of fields in a table that **references the primary key of another table**, establishing a relationship between the two tables.

In this example:

`clientNo` in the Viewing table is a foreign key

This foreign key references the primary key `clientNo` in the Client table

This means each record in the Viewing table is associated with a client in the Client table

Foreign Key

Characteristics of foreign keys:

- 1.Referential integrity: Foreign key values must exist in the referenced table's primary key or be NULL
- 2.Relationship creation: Creates links between tables, allowing us to query across multiple tables
- 3.Cascade operations: When data in the primary key table changes, related data in the foreign key table can be updated accordingly

Relationship Between Them

In this example, the **Client** and **Viewing** tables are connected through a primary-foreign key relationship:

- One client (uniquely identified by clientNo in the Client table) may have multiple viewing records (in the Viewing table)
- But each viewing record belongs to only one client

This is a typical one-to-many relationship. Through JOIN operations, we can query information like "which client viewed which properties."

This **primary-foreign key design** is a core concept in relational databases. It ensures data integrity and consistency while allowing us to organize and query data according to real-world relationship structures.

Challenge 16: Sorting a join

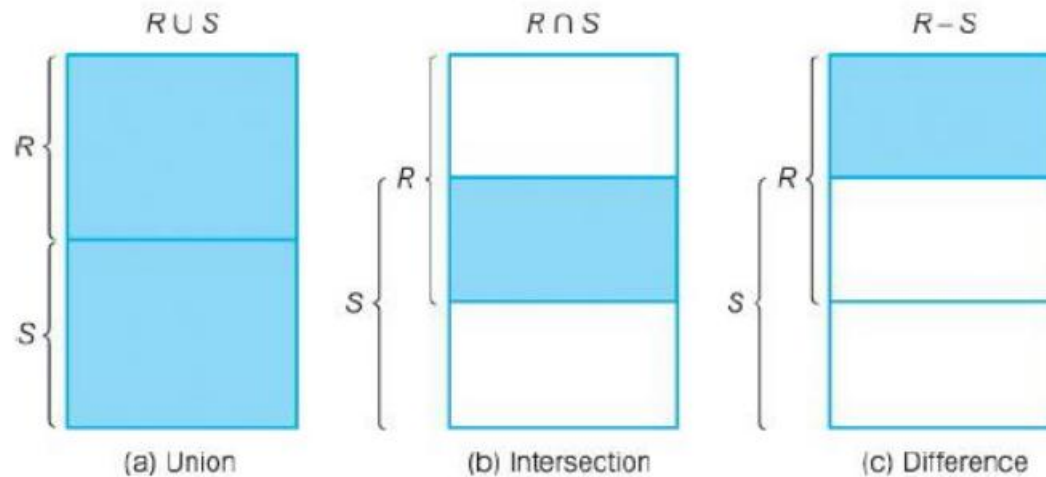
For each branch office, list the staff numbers and names of staff who manage properties and the properties that they manage.

```
SELECT s.branchNo, s.staffNo, fName, Lname, propertyNo  
FROM Staff s, PropertyForRent p  
WHERE s.staffNo = p.staffNo  
ORDER BY s.branchNo, s.staffNo, propertyNo;
```

Part 8 Combining Result Tables (UNION, INTERSECT, EXCEPT)

Set Operations Overview

- SQL supports three standard set operations:
- UNION - All rows from either table A or table B or both
- INTERSECT - Only rows common to both tables A and B
- EXCEPT (Difference) - Rows in table A but not in table B



Part 8 Combining Result Tables (UNION, INTERSECT, EXCEPT)

Union Compatibility Requirements

- Tables must have the same structure to be combined
- Must contain the same number of columns
- Corresponding columns must have matching data types and lengths
- User must ensure corresponding columns have semantically compatible values

Practical Considerations

- Not sensible to combine columns with different meanings
- Example: combining staff age with property room count is inappropriate
- Semantic compatibility is user's responsibility, even when data types match

Basic Syntax for SQL Set Operations

1. UNION

SELECT column1, column2, ... **FROM** table1

UNION

SELECT column1, column2, ... **FROM** table2;

The number of column you selected must be same!!

Basic Syntax for SQL Set Operations

3.DIFFERENCE (EXCEPT)

SELECT column1, column2, ... **FROM** table1

EXCEPT

SELECT column1, column2, ... **FROM** table2;

The number of column you selected must be same!!

Basic Syntax for SQL Set Operations

2. INTERSECTION (INTERSECT)

SELECT column1, column2, ... **FROM** table1

INTERSECT

SELECT column1, column2, ... **FROM** table2;

The number of column you selected must be same!!

Important requirements for all these operations

1. Both queries must return the same number of columns
2. Corresponding columns must have compatible data types
3. The column names from the first query will be used as the column names for the result set

Challenge 17: Use of UNION

Construct a list of all cities where there is either a branch office or a property.

Challenge 17: Use of UNION

Construct a list of all cities where there is either a branch office or a property.

```
SELECT city  
FROM Branch  
WHERE city IS NOT NULL  
UNION  
SELECT city  
FROM PropertyForRent  
WHERE city IS NOT NULL
```

Challenge 18: Use of INTERSECT

Construct a list of all cities where there is both a branch office and a property.

Challenge 18: Use of INTERSECT

Construct a list of all cities where there is both a branch office and a property.

```
SELECT city FROM Branch
```

```
INTERSECT
```

```
SELECT city FROM Property;
```

Challenge 19: Use of EXCEPT

Construct a list of all cities where there is a branch office but no properties.

```
SELECT city  
FROM Branch  
EXCEPT  
SELECT city  
FROM PropertyForRent;
```

```
SELECT DISTINCT Branch.city  
FROM Branch  
LEFT OUTER JOIN PropertyForRent ON Branch.BranchNo =  
PropertyForRent.BranchNo  
WHERE PropertyForRent.BranchNo IS NULL;
```